

MODELS UNDER CONTROL

Implémentation de contrôleurs dans des systèmes cyber-physiques (BIP)

Auteurs :	Encadrantes :
DELACROIX Luc	Olga KOUCHNARENKO
BARTHOD Julien	Anna GALLONE
ROYER Lucas	

Master 2 ISL - Université Marie et Louis Pasteur

Présentation de projet semestriel

24 février 2026

UNIVERSITÉ
MARIE & LOUIS
PASTEUR

Sommaire

- 1 Introduction et Fondations
- 2 Axe 1 : Contrôle Classique (SISO)
- 3 Axe 2 : Contrôle Prédictif (MPC)
- 4 Axe 3 : Extension au MIMO
- 5 Conclusion

Contexte : Systèmes Cyber-Physiques (CPS)

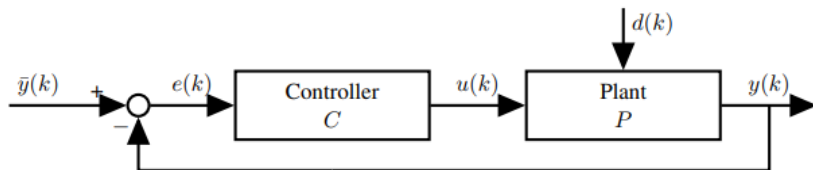
Les systèmes critiques font face à des perturbations environnementales causant des écarts entre le comportement attendu et le comportement réel.

Objectifs du projet :

- Intégrer des modules de correction (Contrôleurs) dans le framework rigoureux BIP (Behavior Interaction Priority).
- Créer une bibliothèque de composants réutilisables en C++.
- Implémenter et valider 3 approches : SISO (PID), Prédictif (MPC), et MIMO (LQR).

Software Engineering Meets Control Theory

Principe fondamental (*Filieri et al., 2015*) : Les approches logicielles traditionnelles peinent face à l'incertain. La solution est d'appliquer une boucle de rétroaction au logiciel.



Architecture d'une boucle de rétroaction appliquée au logiciel

Théorie du Contrôle (*Goodwin et al., 2001*)

- **Stratégie Réactive (SISO/PID) :**
 - Corrige l'erreur mesurée dans le passé.
 - Limites : Réagit aux perturbations après leur apparition.
- **Stratégie Basée Modèle (MPC/LQR) :**
 - Anticipe le futur grâce à la modélisation mathématique du procédé.
 - Optimise la commande en respectant des contraintes strictes.
 - Permet la gestion des systèmes couplés (MIMO).

Organisation et Architecture Logicielle (BIP)

Outils de collaboration :

- **GitHub** : Gestion de version C++/BIP.
- **Discord & Notion** : Suivi, résolution technique et base de connaissances.

Intégration dans l'architecture modulaire BIP :

- **Atomes** : *Blinky Block* (simule le matériel) et *Contrôleur* (notre logique C++).
- **Connecteurs** : Synchronisent les mesures et les commandes.
- **Priorités** : Garantissent le déterminisme global.

Approche SISO : Architecture et Contrôleur PID

Une Architecture Générique (C++) :

- `IControllerSISO` : Interface commune facilitant l'ajout de futurs contrôleurs.
- `ControllerBIPInterface` : Wrappers permettant de traduire nos objets C++ dans l'environnement fonctionnel de BIP.

Le Contrôleur PID (Proportionnel-Intégral-Dérivé) :

- **P** : Réaction immédiate à l'erreur ($K_p \cdot e(t)$).
- **I** : Annulation de l'erreur statique via accumulation temporelle.
- **D** : Amortissement dynamique limitant les oscillations.
- *Gestion des limites* : La commande est bornée (u_{min}, u_{max}) pour simuler la saturation physique.

Résultats et Validation (PID)

Test 1 : Environnement numérique C++

- Modèle du 1er ordre avec bruit de capteur et perturbations variables.
- Analyse de la robustesse.

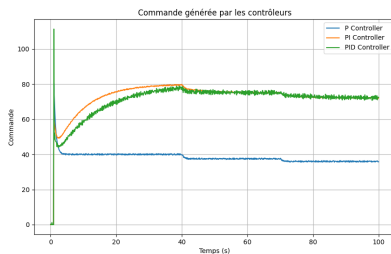
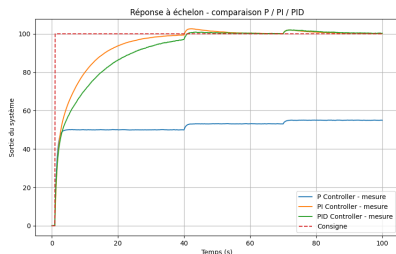


Figure – Résultats tests P/PI/PID C++

Résultats et Validation (PID)

Test 2 : Applaudimètre modifié

- Mesure de l'intensité des applaudissements
- Allumage des LED par rapport à l'intensité
- Correction des applaudissement grâce au PID

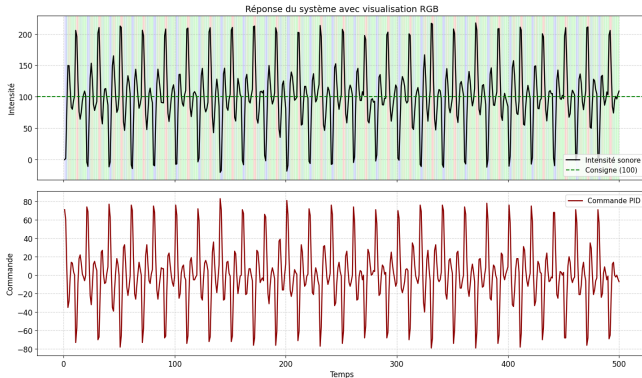


Figure – Résultats test Applaudimètre

Principe du MPC (Model Predictive Control)

Concept clé : L'anticipation proactive (Horizon Glissant).

Fonctionnement :

- **Modèle interne** : Prédit l'évolution temporelle ($y_{k+1} = a \cdot y_k + b \cdot u_k$).
- **Fonction de coût (J)** : Calcule le meilleur compromis sur N pas entre précision (cible) et économie d'effort de l'actionneur.

Implémentation et Sécurité (Safety Check)

Architecture C++ Modulaire :

- IControllerMPC
- MPCController
- ControllerBIPInterface.

Gestion des Contraintes en 3 étapes :

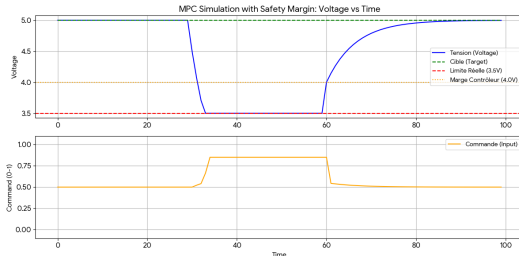
- ➊ **Prédiction idéale** : Calcul analytique de la commande parfaite.
- ➋ **Détection et Sauvetage** : Si la trajectoire prédite viole la sécurité, calcul d'une commande d'urgence pour *coller* à la limite :

$$u_{sauvetage} = \frac{Limite - a \cdot y_k}{b}$$

- ➌ **Saturation** : Diminution pour respecter la puissance de l'actionneur.

Résultats MPC

Scénario : Cible à 5.0V, limite stricte de sécurité à 3.5V (marge interne à 4.0V), et forte perturbation de 0.5V à $t = 30$.



Analyse des 3 phases :

- **Régime permanent** : Stabilité parfaite à 5.0V.
- **Mode survie** : Chute bloquée *précisément* à la limite de 3.5V (optimisation par le moindre effort sous contrainte).
- **Récupération** : Remontée propre dès la fin de la perturbation.

Du SISO au MIMO : gérer des systèmes plus complexes

Jusqu'ici en SISO

- Une entrée agit sur une seule sortie
- Chaque variable est régulée séparément

Dans un système complexe

- Plusieurs entrées et sorties
- Les variables sont couplées entre elles
- Une action u peut influencer plusieurs composantes de x

On passe alors au cadre MIMO :

$$x_{k+1} = Ax_k + Bu_k$$

- x_k : vecteur d'états
- u_k : vecteur d'entrées
- A : dynamique du système
- B : effet des actionneurs

Principe du LQR (Linear Quadratic Regulator)

Le LQR consiste à minimiser le critère :

$$J = \sum_{k=0}^{\infty} (x_k^T Q x_k + u_k^T R u_k)$$

On cherche à corriger l'erreur tout en évitant une commande trop agressive.

- Q : pondère l'importance des états
- R : pondère l'effort appliqué par les actionneurs

Loi de commande obtenue :

$$u_k = -K x_k$$

où K est une matrice de gain calculée par l'équation de Riccati.

Scénario de test MIMO : drone dans un plan

Objectif : atteindre une position cible, avec ou sans vent simulé.

État : $x = [p_x, p_y, v_x, v_y]^T$

Commande : $u = [u_x, u_y]^T$

Modèle du système :

$$A = \begin{bmatrix} 1 & 0 & dt & 0 \\ 0 & 1 & 0 & dt \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} B = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ dt & 0 \\ 0 & dt \end{bmatrix}$$

- A : la position intègre la vitesse
- B : les forces agissent sur les vitesses

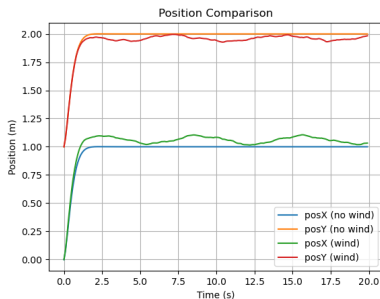
Pondérations LQR :

$$Q = \begin{bmatrix} 10 & 0 & 0 & 0 \\ 0 & 10 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} R = \begin{bmatrix} 0.1 & 0 \\ 0 & 0.1 \end{bmatrix}$$

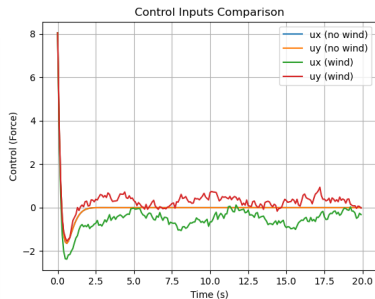
- Q : on pénalise plus la position que la vitesse
- R : on évite une commande trop agressive

Résultats du test LQR (Drone 2D)

Scénario : partir de $(0, 1)$ pour atteindre $(1, 2)$ avec ou sans perturbation.



Évolution des positions



Forces appliquées (u_x , u_y)

Bilan global : une bibliothèque de contrôle intégrée dans BIP

Objectif initial : Intégrer des contrôleurs issus de la théorie du contrôle dans une architecture logicielle rigoureuse (BIP).

Réalisations :

- Implémentation de 3 stratégies complémentaires : PID, MPC, LQR.
- Conception d'une architecture C++ modulaire et réutilisable.
- Intégration via des Wrappers compatibles BIP.
- Validation par simulation sur plusieurs systèmes représentatifs.
- Rédaction de documentation via un outil dédié.

Validation expérimentale : diversité des scénarios

Nombre de cas étudiés :

- 4 tests SISO (P, PI, PID et Applaudimètre BIP)
- 1 test MPC avec différentes perturbations
- 1 test MIMO avec perturbation ou non

Perturbations intégrées :

- Bruit de mesure (capteur imparfait)
- Perturbations aléatoires, constantes et renforcées
- Charge imprévue sévère
- Simulation de bourasques de vent
- Changement des paramètres

Peu de tests, mais des scénarios variés et exigeants.

Court terme :

- Tests sur matériel réel (Blinky Blocks).
- Étude des latences et imprécisions capteurs.

Moyen terme :

- Multiplication des scénarios extrêmes.
- Extension à d'autres contrôleurs avancés.

Vers des systèmes cyber-physiques autonomes, sûrs et adaptatifs.

Merci de votre attention

Questions ?